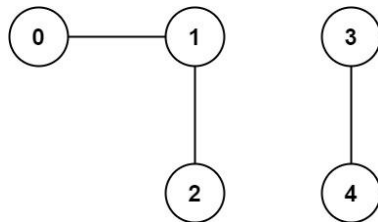


Q1 - Number of connected components

Link - <https://leetcode.com/problems/number-of-connected-components-in-an-undirected-graph>

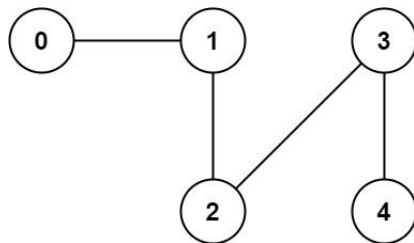
You have a graph of n nodes. You are given an integer n and an array `edges` where `edges[i] = [ai, bi]` indicates that there is an edge between a_i and b_i in the graph.

Return the number of connected components in the graph.



Example 1:

Input: $n = 5$, `edges = [[0,1],[1,2],[3,4]]` **Output:** 2



Example 2:

Input: $n = 5$, `edges = [[0,1],[1,2],[2,3],[3,4]]` **Output:** 1

Constraints:

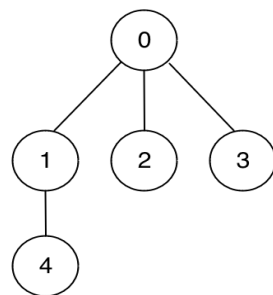
- $1 \leq n \leq 2000$
- $1 \leq \text{edges.length} \leq 5000$
- `edges[i] = [ai, bi]`
- $a_i \neq b_i$
- There are no repeated edges.

Q2 - Graph Valid Tree

Link - <https://leetcode.com/problems/graph-valid-tree/>

Given n nodes labeled from 0 to $n - 1$ and a list of undirected edges (each edge is a pair of nodes), write a function to check whether these edges make up a valid tree.

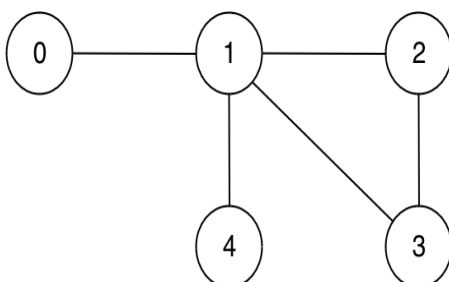
Example 1:



Input: $n = 5$, `edges = [[0,1],[0,2],[0,3],[1,4]]`

Output: true

Example 2:



Input: $n = 5$, `edges = [[0,1],[1,2],[2,3],[1,3],[1,4]]`

Output: false

Note: Assume that no duplicate edges will appear in edges. Since all edges are undirected, `[0, 1]` is the same as `[1, 0]` and thus will not appear together in edges.

Q3 - Given an m x n grid of characters board and a string word, return true if word exists in the grid.

Link - <https://leetcode.com/problems/word-search/>

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Example 1:

A	B	C	E
S	F	C	S
A	D	E	E

Input:

board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]],
word = "ABCCED"

Output: true

Example 2:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board =

[[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]],
word = "SEE"

Output: true

Example 3:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board =

[[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]],
word = "ABCB"

Output: false

Constraints:

- m == board.length
- n = board[i].length
- 1 <= m, n <= 6
- 1 <= word.length <= 15

Board and word consist of only lowercase and uppercase English letters.